



50. Pow(x, n).

Medium

Topics

Companies

Implement `pow(x, n)`, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$

Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$

Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$

Output: 0.25000

Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

$-100.0 < x < 100.0$

$-2^{31} \leq n \leq 2^{31}-1$

n is an integer.

Either x is not zero or $n > 0$.

$-10^4 \leq x^n \leq 10^4$

```
class Solution {
public double myPow(double a, int b) {
double res = 1;
while (b > 0) {
if ((b & 1) == 1) {
res = (res * a) % (1e9 + 7);
}
a = (a * a) % (1e9 + 7);
b /= 2;
}
return res;
}
}
```

Your code is doing modular exponentiation, but this problem wants real x^n with doubles, including negative exponents, and without any modulo.^[1] ^[2]

What is wrong in your code

Your current code:

```
class Solution {
    public double myPow(double a, int b) {
        double res = 1;
        while (b > 0) {
            if ((b & 1) == 1) {
                res = (res * a) % (1e9 + 7);
            }
            a = (a * a) % (1e9 + 7);
            b /= 2;
        }
        return res;
    }
}
```

Problems:

- Uses `% (1e9 + 7)` which is for integer modular exponentiation, not for this problem.^[2]
- Does not handle negative `n` at all (loop `while (b > 0)` just skips when `b < 0`).^[2]
- Does not handle `n == 0` explicitly, although result should be `1.0`.^[2]
- Possible overflow if you try to negate `INT_MIN` directly; must convert exponent to `long`.^[3] ^[2]

Correct iterative binary exponentiation (Java)

Use exponentiation by squaring with doubles and handle negative `n`:

```
class Solution {
    public double myPow(double x, int n) {
        long N = n;           // promote to long to safely negate INT_MIN
        if (N < 0) {
            x = 1.0 / x;      // x^(-n) = (1/x)^n
            N = -N;
        }

        double res = 1.0;
        double base = x;

        while (N > 0) {
            if ((N & 1L) == 1L) { // if current bit set, multiply into result
                res *= base;
            }
            base *= base;        // square base
            N >>= 1;            // shift exponent
        }
    }
}
```

```

        return res;
    }
}

```

This:

- Runs in $O(\log |n|)$ time by binary exponentiation. [\[3\]](#) [\[4\]](#)
- Correctly handles negative exponents by inverting x and making n positive. [\[5\]](#) [\[2\]](#)
- Avoids `int` overflow by using `long N = n` before negation. [\[3\]](#) [\[2\]](#)

How it works (brief)

- If $n < 0$, convert to $x = 1/x$ and $N = -n$, so you always compute a non-negative exponent. [\[3\]](#)
- Represent N in binary and process bits:
 - When the current bit is 1, multiply `res *= base`.
 - Square base each step and shift N right by 1. [\[6\]](#) [\[3\]](#)

This matches the intended solution for LeetCode 50 Pow(x, n). [\[1\]](#) [\[2\]](#)

✱✱

Binary exponentiation is only to find a power B? Or can we find any other things?

Binary exponentiation computes any associative operation raised to a power in $O(\log n)$ time, not just numerical powers like a^b . [\[22\]](#) [\[23\]](#)

Core Applications

Matrix Powering: Compute matrix^n for Fibonacci (n -th term via $[,]^n$) or graph paths (adjacency matrix^n counts walks). [\[23\]](#) [\[24\]](#) [\[22\]](#)

Modular Arithmetic: Fast $a^b \bmod m$ prevents overflow in competitive programming (e.g., counting ways $\bmod 10^9+7$). [\[25\]](#) [\[22\]](#)

Polynomial Multiplication: Raise polynomial to power n using FFT or direct matrix form. [\[22\]](#)

Advanced Uses

Large Multiplications: Compute $a * b \bmod m$ when $a*b$ overflows by treating as $a^{(\log b)}$ with bit additions. [\[25\]](#) [\[22\]](#)

Functional Powers: Compose function f , n times: $f^n(x) = f(f(\dots f(x)\dots))$ using binary exponentiation on function application. [\[22\]](#)

Group Operations: Any semigroup operation (matrix mult, string concat, set union) where $(ab)c = a(bc)$. [\[22\]](#)

The key requirement: the binary operation must be associative. This makes binary exponentiation a universal pattern for repeated application optimization. [\[23\]](#) [\[22\]](#)

Can I get any template that can be used for binary exponentiation?

Binary exponentiation templates work for any associative operation (multiplication, matrix mult, etc.) in $O(\log n)$ time. [\[33\]](#) [\[34\]](#)

Integer Power (Modular)

```
public long modPow(long a, long b, long mod) {
    long res = 1;
    a %= mod;
    while (b > 0) {
        if ((b & 1) == 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}
```

Use: `modPow(2, 10, 1000000007)` → 1024% MOD [\[34\]](#) [\[33\]](#)

Double Power (LeetCode 50)

```
public double myPow(double x, int n) {
    long N = n;
    if (N < 0) {
        x = 1.0 / x;
        N = -N;
    }
    double res = 1.0;
    double base = x;
    while (N > 0) {
        if ((N & 1) == 1) res *= base;
        base *= base;
        N >>= 1;
    }
    return res;
}
```

Handles negative exponents safely. [\[35\]](#) [\[36\]](#)

Matrix Power (Fibonacci)

```
// 2x2 matrix multiplication mod MOD
public long[][] matMul(long[][] a, long[][] b, long mod) {
    int n = a.length;
    long[][] res = new long[n][n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (int k = 0; k < n; k++)
                res[i][j] = (res[i][j] + a[i][k] * b[k][j]) % mod;
    return res;
}

// Matrix power using binary exponentiation
public long[][] matPow(long[][] mat, long exp, long mod) {
    int n = mat.length;
    long[][] res = new long[n][n];
    for (int i = 0; i < n; i++) res[i][i] = 1; // identity matrix

    while (exp > 0) {
        if ((exp & 1) == 1) res = matMul(res, mat, mod);
        mat = matMul(mat, mat, mod);
        exp >>= 1;
    }
    return res;
}
```

Fibonacci n-th term: `matPow({{1,1},{1,0}}, n, MOD)[^3_1][^3_0]` [\[33\]](#)

Generic Template (Any Associative Op)

```
public T binPow(T base, long exp, BinaryOperator<T> op, T identity) {
    T res = identity;
    while (exp > 0) {
        if ((exp & 1) == 1) res = op.apply(res, base);
        base = op.apply(base, base);
        exp >>= 1;
    }
    return res;
}
```

Usage examples:

```
// Integer multiplication
binPow(2L, 10L, (a,b) -> (a*b)%MOD, 1L);

// Custom operation
binPow("ab", 3L, (a,b) -> a+b, "");
```

These templates cover 95% of competitive programming use cases. Always use `long` for exponents to handle `INT_MIN`. [\[37\]](#) [\[34\]](#) [\[33\]](#)

1. <https://leetcode.com/problems/powx-n/>
2. <https://www.hellointerview.com/community/questions/pow-x-n/cm5eh7nrh04oe838ow63ufjn9>
3. <https://algo.monster/liteproblems/50>
4. <https://takeuforward.org/data-structure/implement-powxn-x-raised-to-the-power-n>
5. <https://algomap.io/question-bank/powx-n>
6. [https://github.com/doocs/leetcode/blob/main/solution/0000-0099/0050.Pow\(x,n\)/README_EN.md](https://github.com/doocs/leetcode/blob/main/solution/0000-0099/0050.Pow(x,n)/README_EN.md)
7. <https://www.youtube.com/watch?v=l0YC3876qyg>
8. <https://leetcode.com/problems/powx-n/discuss/1246261/simple-java-recursion-solution-faster-than-100>
9. <https://stackoverflow.com/questions/72414111/why-does-java-integer-act-erroneously-at-values-near-231-1/72414555>
10. <https://www.youtube.com/watch?v=snOaKR2xgZg>
11. <https://www.youtube.com/watch?v=GyL7FJn0gso>
12. https://www.linkedin.com/posts/balanikhilesh_100daysofcode-placementprep-100daysofcode-activity-7365783579605315584-zzLw
13. <https://www.youtube.com/watch?v=1TtWF2ZLNzk>
14. https://jvndb.jvn.jp/search/index.php?mode=_vulnerability_search_IA_VulnSearch&lang=en&vendor=&product=&datePublicFromMonth=&datePublicFromYear=&datePublicToMonth=&datePublicToYear=&dateLastPublishedFromMonth=&dateLastPublishedFromYear=&dateLastPublishedToMonth=&dateLastPublishedToYear=&cwe=&searchProductId=&skey=d1&pageNo=20
15. <https://www.youtube.com/watch?v=g9YQyYi4lQQ>
16. http://archive.org/stream/mademoiselledema00gaute/mademoiselledema00gaute_djvu.txt
17. <https://web.eecs.utk.edu/~jplank/topcoder-writeups/Leetcode/Powx-N/index.html>
18. https://www.youtube.com/watch?v=wxQ9ig_Nd8s
19. <http://www.cplusplus.com/reference/valarray/pow/>
20. <https://www.youtube.com/watch?v=vhzIOc0lITs>
21. https://www.linkedin.com/posts/ritik-gupta-33b8b3292_java-leetcode-exponentiationbysquaring-activity-7313598662842953730-ixDc
22. <https://cp-algorithms.com/algebra/binary-exp.html>
23. <https://www.freecodecamp.org/news/binary-exponentiation-algorithm-explained-with-examples/>
24. <http://www.cplusplus.com/reference/valarray/pow/>
25. <https://www.geeksforgeeks.org/competitive-programming/binary-exponentiation-for-competitive-programming/>
26. <https://codeforces.com/blog/entry/147272>
27. <https://dev.to/dochan/binary-exponentiation-5bfn>
28. <https://algcademy.com/blog/practical-guide-to-modular-exponentiation-efficient-power-calculation-in-cryptography/>
29. https://en.wikipedia.org/wiki/Exponentiation_by_squaring

30. https://www.linkedin.com/posts/sujalsharma_dsa-modularexponentiation-binaryexponentiation-activity-7341179615031263232-n5gc
31. <https://www.youtube.com/watch?v=L-Wzglnm4dM>
32. <https://stackoverflow.com/questions/8771713/efficient-exponentiation-for-huge-numbers-im-talking-googols>
33. <https://cp-algorithms.com/algebra/binary-exp.html>
34. <https://www.geeksforgeeks.org/competitive-programming/binary-exponentiation-for-competitive-programming/>
35. <https://leetcode.com/problems/powx-n/>
36. <https://www.hellointerview.com/community/questions/pow-x-n/cm5eh7nrh04oe838ow63ufjn9>
37. <http://www.cplusplus.com/reference/valarray/pow/>